Advanced Mid-Term Project
AdFlow Pro

Sponsored Listing Marketplace with Moderation, Scheduling, Payment Verification, Analytics, and External Media Normalization

Instructor-Ready Project Brief • MERN / Next.js + Vercel + Supabase

Course Level	Advanced MERN Stack / Full-Stack Web Development
Suggested Duration	4–6 weeks (mid-term implementation window)
Team Size	2–4 students per group
Primary Goal	Build a production-style classified ads workflow platform

This document defines the full scope, architecture, database, APIs, workflow, rubric, and viva guidance for the project.

1. Project Overview

AdFlow Pro is an advanced moderated ads marketplace where clients submit paid listings, moderators review content, administrators verify payments, and approved ads go live for a limited package-based duration. The platform is intentionally designed as a real-world workflow system rather than a simple CRUD website.

Core Concept

- Only approved ads are visible publicly.
- Media is stored as external URLs only; no local image upload is required.
- Payment proof can be captured using transaction details and optional external screenshot URLs.
- Packages control ranking, duration, visibility, and featured placement.
- Automation handles publishing, expiry, notifications, and health checks.

Recommended Tech Stack

- Frontend: Next.js or React
- Backend: Node.js + Express or Next.js API routes
- Database: Supabase Postgres
- Authentication: JWT or Supabase Auth
- Deployment: Vercel (frontend / serverless APIs) with scheduled cron jobs
- Validation: Zod / Joi / express-validator
- UI: Tailwind CSS or Material UI

2. Learning Outcomes

- Design multi-role systems with role-based access control (RBAC).
- Build workflow-driven backend logic beyond basic CRUD.
- Model relational data in Postgres for business applications.
- Implement package rules, status transitions, ranking, and expiry logic.
- Handle third-party media URLs and normalized thumbnail generation.
- Create dashboards, metrics, search, filtering, and moderation panels.
- Use scheduled jobs for publishing, expiration, reminders, and DB heartbeat.
- Deploy a production-style full-stack app on Vercel + Supabase.

3. User Roles and Permissions

Role	Main Responsibility	Key Permissions	Visibility
Client	Create and manage own ads	Register, login, submit ad, submit payment proof, view status, edit draft	Own dashboard only
Moderator	Review content quality and policy fit	Review ads, flag suspicious media, reject with reason, add internal notes	Moderation panel

Admin	Payment verification and publishing control	Verify payments, approve/reject, schedule, feature ads, manage users	Admin dashboard
Super Admin	System-level control	Manage packages, settings, categories, featured rules, reports, health metrics	Full system access

4. Public Platform Modules

1. Landing Page with hero section, packages, featured ads, recent ads, trust badges, and learning question widget.
2. Explore Ads page with search, category filters, city filters, sort options, pagination, and active-only view.
3. Ad Detail page with normalized media preview, seller summary, package badge, expiry date, and report button.
4. Category pages and city pages for browsable taxonomy-driven navigation.
5. Package page showing listing durations, benefits, and homepage placement rules.
6. FAQ, Contact, Terms, Privacy, and platform usage policy pages.

5. Internal Workflow Modules

Client Dashboard

- Create ad draft
- Submit listing request
- Select package
- Submit payment details
- Track review status
- View active / expired / rejected ads
- Receive notifications

Moderator Dashboard

- Review submitted ads
- Check title, description, category, and media URLs
- Flag suspicious content or duplicate listings
- Add moderation notes
- Reject or move to payment stage

Admin Dashboard

- Verify payment records
- Approve / reject / schedule publishing
- Mark featured ads

- Monitor active and expiring listings
- Open analytics and system reports
- Manage users, categories, packages, and settings

6. Ad Lifecycle Workflow

Stage	Who Triggers It	Required Condition	Next Possible State	Visible Publicly?
Draft	Client	Ad form started	Submitted	No
Submitted	Client	Validation passed	Under Review	No
Under Review	Moderator	Content check initiated	Payment Pending / Rejected	No
Payment Pending	Client	Package selected	Payment Submitted	No
Payment Submitted	Client	Transaction details provided	Payment Verified / Rejected	No
Payment Verified	Admin	Proof accepted	Scheduled / Published	No
Scheduled	Admin	Future publish date set	Published	No
Published	System/Admin	Publish date reached	Expired / Archived	Yes
Expired	System	End date passed	Archived / Renewed	No

7. External Media Strategy (No Local Uploads)

To avoid storage limitations, the platform stores only text-based media references in the database. Students must normalize, validate, and preview these URLs at runtime.

Allowed Media Sources

- GitHub raw image URLs
- Direct public image URLs
- YouTube video URLs with auto-generated thumbnails
- Optional Cloudinary or CDN links if allowed by instructor

Media Normalization Rules

- If source is YouTube, extract video ID and generate thumbnail URL automatically.
- If source is image URL, validate protocol, extension, and allowed domain policy.
- If media fails to load, display a placeholder preview instead of breaking layout.
- Save source_type, original_url, normalized_thumbnail_url, and validation_status.

8. Package Engine

Package	Duration	Homepage Visibility	Featured Weight	Refresh Rule	Suggested Price Logic
Basic	7 days	No	1x	None	Lowest entry plan
Standard	15 days	Category priority	2x	Manual refresh	Balanced option
Premium	30 days	Homepage	3x	Auto refresh	High-value plan

		featured		every 3 days	

Students may extend packages with badges, category limits, multiple ad quotas, or support priority.

9. Ranking and Visibility Logic

The public ads list should not rely only on newest-first ordering. Students should implement a rank score that combines package strength, freshness, featured status, and optional manual admin boost.

```
rankScore = (featured ? 50 : 0) + (packageWeight * 10) + freshnessPoints + adminBoost + verifiedSellerPoints
```

- Featured ads appear before normal ads.
- Premium package increases priority.
- Freshly published ads gain temporary visibility boost.
- Expired ads must never appear in public results.
- Search and filters apply after active-status screening.

10. Database Design (Suggested Core Tables)

Table	Purpose	Important Fields
users	Account and role identity	id, name, email, password_hash, role, status, created_at
seller_profiles	Public seller metadata	user_id, display_name, business_name, phone, city, is_verified
packages	Package definitions	id, name, duration_days, weight, is_featured, price
categories	Listing classification	id, name, slug, is_active
cities	Location taxonomy	id, name, slug, is_active
ads	Main listing record	id, user_id, package_id, title, slug, category_id, city_id, description, status, publish_at, expire_at
ad_media	Media normalization	ad_id, source_type, original_url, thumbnail_url, validation_status
payments	Payment proof	ad_id, amount, method, transaction_ref, sender_name, screenshot_url, status
notifications	In-app alerts	user_id, title, message, type, is_read, link
audit_logs	Traceability	actor_id, action_type, target_type, target_id, old_value, new_value, created_at
ad_status_history	Workflow tracking	ad_id, previous_status, new_status, changed_by, note, changed_at
learning_questions	Keep-alive demo content	question, answer, topic, difficulty, is_active
system_health_logs	DB and cron monitoring	source, response_ms, checked_at, status

11. REST API Design (Suggested)

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Create account	Public
POST	/api/auth/login	Authenticate user	Public
GET	/api/packages	Fetch packages	Public
GET	/api/ads	Browse active ads	Public
GET	/api/ads/:slug	Open ad detail	Public
POST	/api/client/ads	Create ad draft	Client
PATCH	/api/client/ads/:id	Edit own ad	Client
POST	/api/client/payments	Submit payment proof	Client
GET	/api/client/dashboard	Get own listings and statuses	Client
GET	/api/moderator/review-queue	Open moderation queue	Moderator
PATCH	/api/moderator/ads/:id/review	Approve content or reject	Moderator
GET	/api/admin/payment-queue	Open payment verification queue	Admin
PATCH	/api/admin/payments/:id/verify	Verify or reject payment	Admin
PATCH	/api/admin/ads/:id/publish	Publish or schedule ad	Admin
GET	/api/admin/analytics/summary	Dashboard metrics	Admin
GET	/api/questions/random	Learning question widget	Public
GET	/api/health/db	Database heartbeat check	System
POST	/api/cron/publish-scheduled	Publish due ads	System
POST	/api/cron/expire-ads	Expire outdated ads	System

12. Business Rules and Validation

- Only approved and non-expired ads are public.
- Clients can edit drafts, but published ads require admin review for critical changes.
- A payment record must exist before an ad can move to published state.
- A scheduled ad becomes visible only when publish_at is reached.
- When expire_at passes, ad status changes to expired automatically or is hidden from queries.
- Duplicate transaction references should be blocked or flagged.
- Media URLs must be sanitized and normalized before storage.
- All state changes should write to audit_logs and ad_status_history.

13. Scheduled Jobs / Automation

- Publish scheduled ads every hour or on an instructor-defined interval.
- Expire outdated ads daily.
- Send expiring-soon notifications 48 hours before expiry.
- Insert DB heartbeat logs for monitoring and Supabase activity.
- Refresh analytics snapshots if a materialized reporting table is used.

14. Analytics Dashboard Requirements

Metric Group	Required KPIs	Example Visual
Listings	Total ads, active ads, pending reviews, expired ads	Summary cards
Revenue	Revenue by package, verified payments, monthly total	Bar chart
Moderation	Approval rate, rejection rate, flagged ads	Donut or stacked bar
Taxonomy	Ads by category, ads by city	Horizontal bar chart
Operations	Scheduled jobs success, DB heartbeat, failed validations	Status cards / table

15. Suggested Frontend Pages

- Home
- Explore Ads
- Ad Details
- Packages
- Category Listing
- City Listing
- Client Dashboard
- Moderator Review Queue
- Admin Dashboard
- Analytics Page
- System Health Page
- Login / Register / Forgot Password

16. Folder Structure (Example)

```
client/
  src/
    pages/ or app/
    components/
    features/
    hooks/
    utils/
server/
  src/
    routes/
    controllers/
    services/
    middlewares/
    validators/
    cron/
    db/
shared/
```

```
constants/
schemas/
types/
```

17. Minimum Deliverables

- Working deployed application link
- Source code repository
- Database schema / SQL export
- API documentation or Postman collection
- At least one scheduled automation
- Admin, moderator, and client login flow
- Sample dataset with at least 15–25 ads
- Short project demo video or live demonstration

18. Milestone Plan

Week	Milestone	Expected Output	Weight
1	Planning and schema	Roles, ERD, routes, wireframes	10%
2	Authentication and dashboards	Role-protected panels	15%
3	Ads workflow and packages	Submission to review flow	20%
4	Payments, publishing, and expiry	Verification and lifecycle automation	20%
5	Search, analytics, and QA	Public browsing + metrics	15%
6	Deployment and presentation	Live app + final report	20%

19. Grading Rubric

Criterion	What to Assess	Marks
Architecture Design	Logical structure, scalability, clean separation of concerns	15
Database Design	Quality of schema, relationships, constraints, normalization	15
Authentication & RBAC	Secure access, protected routes, role restrictions	10
Workflow Logic	Correct status transitions, review process, publish/expire handling	15
API Quality	REST clarity, validation, error handling, consistency	10
Frontend UX	Usability, dashboards, responsive design, navigation	10
Analytics & Automation	Metrics, cron jobs, health checks, system behavior	10

Code Quality	Readability, modularity, comments, naming, best practices	5
Deployment & Demo	Successful deployment, functioning links, demonstration quality	5
Presentation / Viva	Understanding, confidence, problem-solving clarity	5

Total Marks: 100

20. Viva Questions

7. Why did you choose Supabase Postgres instead of local file storage for this project?
8. How does the ad lifecycle protect business logic better than a simple approved/rejected model?
9. How did you implement role-based access control across client, moderator, admin, and super admin?
10. How are external media URLs validated and normalized?
11. What happens when an ad reaches its expiry date?
12. How is payment verification modeled in your database and APIs?
13. How does your ranking formula decide which ad appears first?
14. What tables support traceability and debugging in your system?
15. How would you prevent duplicate ads or fake payment submissions?
16. How would you scale this platform for thousands of listings?

21. Extra Credit Ideas

- Saved ads / bookmarks
- Duplicate detection using phone number or content similarity
- Report abuse / spam workflow
- Coupon codes or discount packages
- Seller verification badges
- Email or WhatsApp notification integration
- Materialized views for reporting
- Soft delete and archive recovery

22. Instructor Notes

This project is ideal for advanced students because it tests architecture, workflow thinking, validation, database design, access control, and deployment maturity. It can be assigned as a team project or an individual capstone-style mid-term. The recommended evaluation focus should remain on system thinking and implementation quality rather than visual decoration alone.

23. Final Project Statement

Students are required to build AdFlow Pro as a production-style sponsored listing marketplace where clients submit listings, moderators review content, administrators verify payments, and only

approved ads become visible for a limited package-based duration. The implementation must demonstrate sound architecture, business rules, role-based access, scheduled automation, and deployment readiness.